

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1
«Реализация REST API на основе boilerplate»

Выполнил:

Аплеев Дмитрий

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

На основе boilerplate и результатов ДЗ1 и ДЗ2 реализовать модели, контроллеры и полностью рабочее API.

Вариант 5. Сервис для обмена рецептами и кулинарных блогов.

Ход работы

1. Основа проекта

Взял boilerplate преподавателя express typeorm boilerplate. Используются Express, routing controllers, TypeORM, PostgreSQL, class validator, jsonwebtoken, bcrypt и Swagger.

```
lab1/src/  
  models/           таблицы (User, Recipe, Ingredient, Comment, Like,  
SavedRecipe, Subscription)  
  controllers/      эндпоинты  
  middlewares/      проверка JWT токена  
  config/           настройки и подключение к базе  
  utils/            хеширование и проверка пароля
```

2. Модели

Каждая таблица из ДЗ1 описана классом с декораторами TypeORM. Типы колонок указаны явно. Таблицы в базе создаются сами при запуске.

```
@Entity()  
export class Recipe extends BaseEntity {  
  @PrimaryGeneratedColumn()  
  id: number;  
  
  @Column({ type: 'int' })  
  author_id: number;  
  
  @ManyToOne(() => User, { onDelete: 'CASCADE' })  
  @JoinColumn({ name: 'author_id' })  
  author: User;  
  
  @Column({ type: 'enum', enum: DishType })  
  dish_type: DishType;  
  ...  
}
```

3. Контроллеры

Контроллер	Эндпоинты
auth	регистрация, вход
user	мой профиль, изменение профиля, публичный профиль
recipe	поиск, создание, страница, изменение, удаление рецепта
comment	список, создание, удаление комментария
like	поставить и убрать лайк
saved-recipe	сохранить, убрать, список сохранённых
subscription	подписаться, отписаться, мои подписки

Таблица 1. Контроллеры

4. Вход и права

При регистрации пароль хешируется в `user.subscriber.ts` перед записью в базу. При входе пароль сверяется с хешем и выдаётся JWT токен. Закрытые эндпоинты помечены `@UseBefore(authMiddleware)`. Он проверяет токен и кладёт пользователя в `request.user`. Изменять и удалять рецепт и комментарий может только автор или админ, иначе ответ 403.

```
if (
  recipe.author_id !== request.user.id &&
  request.user.role !== Role.ADMIN
) {
  throw new ForbiddenError('You can delete only your own recipes');
}
```

5. Поиск рецептов

Фильтры по типу блюда, сложности и автору добавляются в запрос, только если переданы. Для поиска по ингредиенту названия хранятся в нижнем регистре и сравниваются через LIKE.

```
if (ingredient) {
  query.andWhere(
    'recipe.id IN (SELECT recipe_id FROM ingredient WHERE name LIKE :ingredient)',
    { ingredient: `%${String(ingredient).toLowerCase()}%` },
  );
}
```

6. Запуск

Приложение запускается командой `npm run dev`, документация открывается по адресу `http://localhost:3000/docs`. Все 21 эндпоинт проверены через Swagger и Postman.

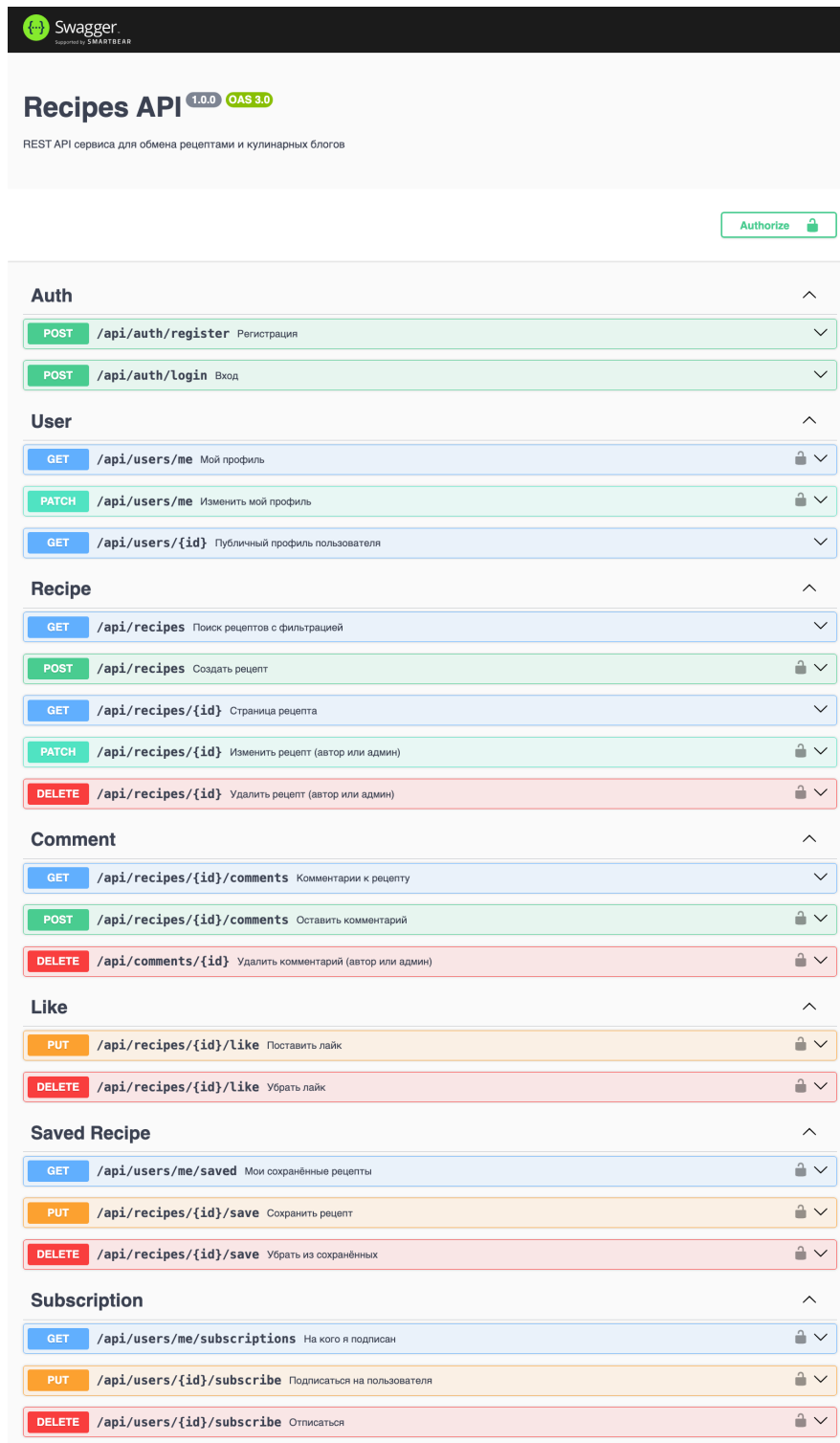


Рисунок 1. API в Swagger

Вывод

На основе boilerplate сделано рабочее REST API сервиса рецептов. Реализованы семь моделей, семь контроллеров, вход по JWT, проверка прав и поиск с фильтрами.